Análise do utilitário echo.c

Marco Mangan

Setembro de 2021

Conteúdo

1 Introdução **2**
1.1 Código-fonte 2
1.2 Idiomas de uma Linguagem de Programação 2
1.3 Blocos de Responsabilidade 2
1.4 Estrutura deste Texto 2

2 Idiomas **2**
2.1 Desvio incondicional 3
2.2 Contador regressivo 3
2.3 Aritmética de ponteiros 3

3 Divisão em Blocos **3**
3.1 Análise do código 3
3.2 Dependências 3
3.3 Blocos propostos 4

4 Cenário principal **5**
4.1 Bloco de Interpretação 7
4.2 Blocos de documentação 7
4.3 Parâmetros de ajuda e versão 7
4.4 Direitos Autorais 7
4.5 Manual do usuário 7
4.6 Preâmbulos 7
4.7 Conversão de base 7
4.8 Nova linha ao final 7
4.9 Configurações 8
4.10 Parâmetros de interpretação 8
4.11 Caracteres interpretados 8

1 Introdução

O utilitário echo apresenta na saída de dados os argumentos recebidos em linha de comando. Este documento apresenta a análise do código-fonte desse utilitário, sendo uma forma de aprender sobre um exemplo real de um programa de software básico (Spinellis 2003).

1.1 Código-fonte

O comando echo disponível em um computador pode ser parte da implementação do utilitário bash. Neste relato, foi analisado o programa independente que é parte da distribuição GNU GNU 2021.

1.2 Idiomas de uma Linguagem de Programação

A programação de software básico em linguagem C desenvolve idiomas, formas particulares de uso da linguagem. Os idiomas são familiares para programadores neste nível de abstração, mas podem ser desconhecidos para programadores de outros níveis, principalmente, programadores de aplicativos.

Os idiomas de software básico são motivados pela otimização do código em uso de memória ou uso do processador. O idioma não é um estilo sintático.

Exemplos de idiomas incluem: (a) uso de desvio incondicional, (b) contador regressivos, e (c) aritmética de ponteiros. São consideradas por outros programadores como "coisas de programador C".

1.3 Blocos de Responsabilidade

O código do programa pode ser agrupado em blocos de acordo com diferentes responsabilidades. A divisão em blocos (*chunks*) é uma estratégia para compreender um programa extenso (Mayrhauser e Vans 1995).

1.4 Estrutura deste Texto

O restante deste documento está organizado como segue. Exemplos dos principais idiomas são apresentados (Seção 2). Em seguida, o principal bloco do programa é apresentado (Seção 4). O programa é dividido em blocos, de acordo com a responsabilidade A listagem completa do programa está ao final deste documento.

2 Idiomas

Foram identificados três idiomas, que diferem dos idiomas utilizados em software aplicativo ou em outras linguagens.

2.1 Desvio incondicional

O comando goto não existe em diversas linguagens de programação. A programação estruturada indica que o uso de desvios ameaça a qualidade e a manutenção de software. Por outro lado, o desvio aparece como uma primitiva de processadores (ex.: JMP).

Foram identificados dois rótulos: not_an_escape e just_echo.

Os argumentos são processados e no caso de um argumento inválido o rótulo just_echo para desconsiderar os parâmetros restantes. Caso todos os parâmetros sejam aceitos, a execução chega ao mesmo rótulo.

O rótulo not_an_escape é utilizado quando um caractere de controle não é identificado, neste caso o caractere é apresentado como uma sequência de duas caracteres.

2.2 Contador regressivo

A valor zero é interpretado como um valor lógico falso. O contador regressivo utiliza essa característica para identificar o final da iteração. O valor de argc é decrementado para controlar o processamento dos argumentos da linha de comando. Com isso, se reduz o número de variáveis e operadores para controlar o laço.

2.3 Aritmética de ponteiros

A entrada do programa echo é realizada por argumentos em linha de comando. Para processar os diversos parâmetros, o endereço base da variável argv é incrementado. Com isso, se evita o custo de calcular o endereço de um elemento indicado por indexação.

3 Divisão em Blocos

3.1 Análise do código

3.2 Dependências

As dependências são formadas por identificadores definidos no código-fonte e também por identificadores externos (Tabela 1). O preâmbulo indica as bibliotecas utilizadas. O projeto GNU utiliza um cabeçalho unificado que inclui outros cabeçalhos como stdlib.h e safe-ctype.h (Figura1).

Início	Fim	Responsabilidade
echo	35	usage()
echo	88	hextobin()
echo	107	main()
echo	24	PROGRAM_NAME
echo	26	AUTHORS

Tabela 1 continua a página anterior

Início	Fim	Responsabilidade
echo	27	proper_name()
echo	31	DEFAULT_ECHO_TO_XPG
assert.h	40	assert()
stdio.h	42	printf()
stdio.h	46	fputs()
system.h	59	HELP_OPTION_DESCRIPTION
system.h	60	VERSION_OPTION_DESCRIPTION
system.h	82	USAGE_BUILTIN_WARNING
system.h	83	emit_ancillary_info
system.h.h	84	exit
stdlib.h (stdlib.h)	111	getenv
system.h	114	STREQ
system.h	121	initialize_main
	122	set_program_name
	123	setlocale
	124	bindtextdomain
	125	textdomain
	127	atexit
system.h (stdlib.h)	134	EXIT_SUCCESS
	138	version_etc
	macro	PACKAGE
	macro	LOCALEDIR
	273	close_stdout
system.h (safe-ctype.h)	273	isxdigit()
sys/types.h	273	size_t
	macro	FALLTHROUGH
stdio.h	273	putchar()

Tabela 1: Dependências internas e externas

3.3 Blocos propostos

O código pode ser dividido em 11 blocos (Tabela 2). Dois blocos controlam a saída de um caractere de nova linha extra ao final da execução do programa.

Início	Fim	Responsabilidade
1	15	Direitos autorais
16	34	Preâmbulos
35	85	Manual do usuário
86	102	Conversão de base
103	106	Nova linha ao final (ver 268)

Tabela 2 continua a página anterior

Início	Fim	Responsabilidade
107	128	Configurações
129	143	Parâmetros de ajuda e versão
144	190	Demais parâmetros
191	259	Caracteres interpretados
260	267	Cenário principal
268	273	Nova linha

Tabela 2: Blocos do programa echo.c

A execução do programa apresenta três caminhos principais: (a) ajuda e versão podem ser apresentados, o programa executa com (b) caracteres de controle da entrada sendo interpretados ou (c) com entrada literal (Figura 2).

4 Cenário principal

O utilitário apresenta 273 linhas. A função principal do utilitário é programada em apenas oito linhas (Código-fonte 1).

O texto utiliza convenções de identação do projeto GNU. Identificadores seguem o padrão de codificação de outras bibliotecas escritas na linguagem C.

Código-fonte 1: Cenário básico, eco literal

```
260       while (argc > 0)
261         {
262           fputs (argv[0], stdout);
263           argc--;
264           argv++;
265           if (argc > 0)
266             putchar (' ');
267         }
```

O bloco principal apresenta cada argumento da linha de comando, separados por um espaço em branco, exceto o último elemento. A aritmética de ponteiros é utilizada para avançar o endereço base de argv. O contador regressivo evita uma variável adicional e serve para controlar a inclusão do separador.

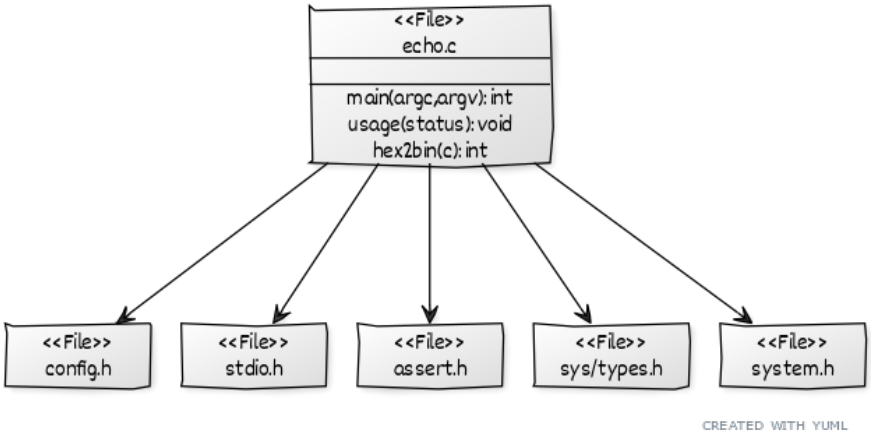

Figura 1: Programa e Bibliotecas

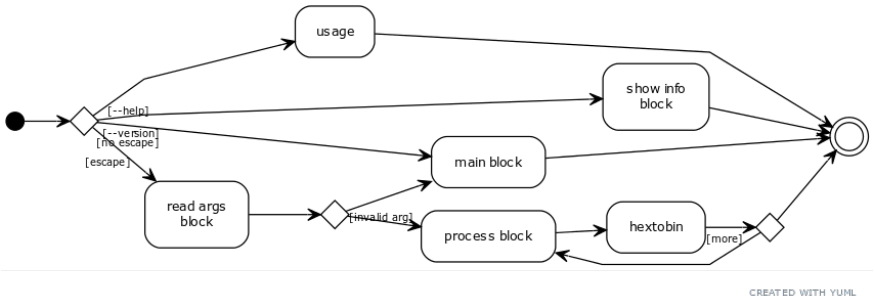

Figura 2: Fluxo de execução

4.1 Bloco de Interpretação

4.2 Blocos de documentação

4.3 Parâmetros de ajuda e versão

4.4 Direitos Autorais

A documentação do sistema é o primeiro bloco. Os principais itens são: (a) licença de uso, (b) evolução do sistema e (c) idade do programa. O programa adota uma licença de software livre específica. A licença permite que o código-fonte seja consultado, o que permite este estudo. O programa é derivado de um trecho de código do utilitário bash. No formato atual, o programa existe desde 1987. Em 2021, o programa acumula mais de três décadas de manutenção e de serviço.

4.5 Manual do usuário

A documentação para o usuário final serve dois propósitos: (a) especificação e (b) manual de usuário.

4.6 Preâmbulos

Brian Jahn Fox estudou no Massachusetts Institute of Techonology em 1986 e escreveu o código do bash em 1989 para o GNU. Fox trabalhou em um sistema de votação em código aberto.

Chet Ramey estudou na Case Western Reserve University e contribuiu para o projeto Bash no GNU.

Ramey e Fox publicaram um livro "Bash: Reference Manual"

Inclusão de cabeçalhos de bibliotecas externas e da biblioteca interna (system.h).

4.7 Conversão de base

Uma rotina isola a conversão de

4.8 Nova linha ao final

Um dos argumentos controla a apresentação de um caractere ao final da saída.

4.9 Configurações

4.10 Parâmetros de interpretação

4.11 Caracteres interpretados

Conclusão

Foram descritos três diferentes idiomas: desvio incondicional, contador regressivo e aritmética de ponteiros.

O código-fonte foi dividido em 11 blocos. O bloco principal apresenta somente oito linhas. O restante do código trata do contexto do projeto, com aspectos legais, documentação e tratamento de caracteres de controle.

A análise foi realizada com a leitura de texto. Em trabalhos futuros, seria necessário utilizar ferramentas para coletar dados e navegar sobre o código-fonte para poder tratar um maior volume de linhas de código.

Referências

GNU (2021). *Código-fonte do utilitário echo.c*. Último acesso em setembro de 2021. URL: `https://git.savannah.gnu.org/cgit/coreutils.git/plain/src/echo.c`.

Mayrhauser, Anneliese e A. Marie Vans (set. de 1995). «Program comprehension during software maintenance and evolution». Em: *Computer* 28, pp. 44–55. DOI: `10.1109/2.402076`.

Spinellis, D. (2003). *Code Reading: The Open Source Perspective*. Effective Software Development Series. Pearson Education. ISBN: 9780672333705. URL: `https://books.google.com.br/books?id=8lYbNfsAVT4C`.

Anexo

Código-fonte 2: Código-fonte no arquivo echo.c, na íntegra

```
/* echo.c, derived from code echo.c in Bash.
   Copyright (C) 1987-2021 Free Software Foundation, Inc.

   This program is free software: you can redistribute it and/or modify
   it under the terms of the GNU General Public License as published by
   the Free Software Foundation, either version 3 of the License, or
   (at your option) any later version.

```

```c
   This program is distributed in the hope that it will be useful,
   but WITHOUT ANY WARRANTY; without even the implied warranty of
   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
   GNU General Public License for more details.

   You should have received a copy of the GNU General Public License
   along with this program.  If not, see <https://www.gnu.org/licenses/>.  */

#include <config.h>
#include <stdio.h>
#include <assert.h>
#include <sys/types.h>
#include "system.h"

/* The official name of this program (e.g., no 'g' prefix).  */
#define PROGRAM_NAME "echo"

#define AUTHORS \
  proper_name ("Brian Fox"), \
  proper_name ("Chet Ramey")

/* If true, interpret backslash escapes by default.  */
#ifndef DEFAULT_ECHO_TO_XPG
enum { DEFAULT_ECHO_TO_XPG = false };
#endif

void
usage (int status)
{
  /* STATUS should always be EXIT_SUCCESS (unlike in most other
     utilities which would call emit_try_help otherwise).  */
  assert (status == EXIT_SUCCESS);

  printf (_("\
Usage: %s [SHORT-OPTION]... [STRING]...\n\
  or:  %s LONG-OPTION\n\
"), program_name, program_name);
  fputs (_("\
Echo the STRING(s) to standard output.\n\
\n\
  -n             do not output the trailing newline\n\
"), stdout);
  fputs (_(DEFAULT_ECHO_TO_XPG
```

```
            ? N_("\
  -e             enable interpretation of backslash escapes (default)\n\
  -E             disable interpretation of backslash escapes\n")
            : N_("\
  -e             enable interpretation of backslash escapes\n\
  -E             disable interpretation of backslash escapes (default)\n")),
          stdout);
  fputs (HELP_OPTION_DESCRIPTION, stdout);
  fputs (VERSION_OPTION_DESCRIPTION, stdout);
  fputs (_("\
\n\
If -e is in effect, the following sequences are recognized:\n\
\n\
"), stdout);
  fputs (_("\
  \\\\      backslash\n\
  \\a      alert (BEL)\n\
  \\b      backspace\n\
  \\c      produce no further output\n\
  \\e      escape\n\
  \\f      form feed\n\
  \\n      new line\n\
  \\r      carriage return\n\
  \\t      horizontal tab\n\
  \\v      vertical tab\n\
"), stdout);
  fputs (_("\
  \\ONNN   byte with octal value NNN (1 to 3 digits)\n\
  \\xHH    byte with hexadecimal value HH (1 to 2 digits)\n\
"), stdout);
  printf (USAGE_BUILTIN_WARNING, PROGRAM_NAME);
  emit_ancillary_info (PROGRAM_NAME);
  exit (status);
}

/* Convert C from hexadecimal character to integer.  */
static int
hextobin (unsigned char c)
{
  switch (c)
    {
    default: return c - '0';
    case 'a': case 'A': return 10;
    case 'b': case 'B': return 11;
```

```
    case 'c': case 'C': return 12;
    case 'd': case 'D': return 13;
    case 'e': case 'E': return 14;
    case 'f': case 'F': return 15;
    }
}

/* Print the words in LIST to standard output.  If the first word is
   '-n', then don't print a trailing newline.  We also support the
   echo syntax from Version 9 unix systems. */

int
main (int argc, char **argv)
{
  bool display_return = true;
  bool posixly_correct = !!getenv ("POSIXLY_CORRECT");
  bool allow_options =
    (! posixly_correct
     || (! DEFAULT_ECHO_TO_XPG && 1 < argc && STREQ (argv[1], "-n")));

  /* System V machines already have a /bin/sh with a v9 behavior.
     Use the identical behavior for these machines so that the
     existing system shell scripts won't barf.  */
  bool do_v9 = DEFAULT_ECHO_TO_XPG;

  initialize_main (&argc, &argv);
  set_program_name (argv[0]);
  setlocale (LC_ALL, "");
  bindtextdomain (PACKAGE, LOCALEDIR);
  textdomain (PACKAGE);

  atexit (close_stdout);

  /* We directly parse options, rather than use parse_long_options, in
     order to avoid accepting abbreviations.  */
  if (allow_options && argc == 2)
    {
      if (STREQ (argv[1], "--help"))
        usage (EXIT_SUCCESS);

      if (STREQ (argv[1], "--version"))
        {
```

```
          version_etc (stdout, PROGRAM_NAME, PACKAGE_NAME, Version, AUTHORS,
                       (char *) NULL);
          return EXIT_SUCCESS;
        }
    }

  --argc;
  ++argv;

  if (allow_options)
    while (argc > 0 && *argv[0] == '-')
      {
        char const *temp = argv[0] + 1;
        size_t i;

        /* If it appears that we are handling options, then make sure that
           all of the options specified are actually valid. Otherwise, the
           string should just be echoed.  */

        for (i = 0; temp[i]; i++)
          switch (temp[i])
            {
            case 'e': case 'E': case 'n':
              break;
            default:
              goto just_echo;
            }

        if (i == 0)
          goto just_echo;

        /* All of the options in TEMP are valid options to ECHO.
           Handle them. */
        while (*temp)
          switch (*temp++)
            {
            case 'e':
              do_v9 = true;
              break;

            case 'E':
              do_v9 = false;
              break;
```

```

            case 'n':
              display_return = false;
              break;
            }

        argc--;
        argv++;
      }

just_echo:

  if (do_v9 || posixly_correct)
    {
      while (argc > 0)
        {
          char const *s = argv[0];
          unsigned char c;

          while ((c = *s++))
            {
              if (c == '\\' && *s)
                {
                  switch (c = *s++)
                    {
                    case 'a': c = '\a'; break;
                    case 'b': c = '\b'; break;
                    case 'c': return EXIT_SUCCESS;
                    case 'e': c = '\x1B'; break;
                    case 'f': c = '\f'; break;
                    case 'n': c = '\n'; break;
                    case 'r': c = '\r'; break;
                    case 't': c = '\t'; break;
                    case 'v': c = '\v'; break;
                    case 'x':
                      {
                        unsigned char ch = *s;
                        if (! isxdigit (ch))
                          goto not_an_escape;
                        s++;
                        c = hextobin (ch);
                        ch = *s;
                        if (isxdigit (ch))
                          {
                            s++;
                            c = c * 16 + hextobin (ch);
```

```
                        }
                      }
                      break;
                    case '0':
                      c = 0;
                      if (! ('0' <= *s && *s <= '7'))
                        break;
                      c = *s++;
                      FALLTHROUGH;
                    case '1': case '2': case '3':
                    case '4': case '5': case '6': case '7':
                      c -= '0';
                      if ('0' <= *s && *s <= '7')
                        c = c * 8 + (*s++ - '0');
                      if ('0' <= *s && *s <= '7')
                        c = c * 8 + (*s++ - '0');
                      break;
                    case '\\': break;

                    not_an_escape:
                    default:  putchar ('\\'); break;
                    }
                  }
                putchar (c);
              }
            argc--;
            argv++;
            if (argc > 0)
              putchar (' ');
          }
      }
    else
      {
        while (argc > 0)
          {
            fputs (argv[0], stdout);
            argc--;
            argv++;
            if (argc > 0)
              putchar (' ');
          }
      }

    if (display_return)
      putchar ('\n');
    return EXIT_SUCCESS;
```

```
}
```